THE DANCING SAND THEOREM — LEAN CERTIFICATE v1

A machine-checked proof, in Lean 4 with Mathlib, of every theorem of «The Dancing Sand Theorem»: the whole 2-part of the sandpile group of the hypercube, the fold law, and their consequences

Rafael Amichis Luengo · Madrid, Spain · tretoef@gmail.com

Certificate v1 · 11 October 2026 · read cold and corrected the same day (§2 item 7)

Companion to *The Dancing Sand Theorem — The 2-part of the sandpile group of the hypercube, for every n*, version 8 (6 October 2026), Zenodo DOI 10.5281/zenodo.23188299. The numbers of sections and results below are those of version 8.

---

0. Verdict

For **every** $n \geq 1$, Lean 4 proves:

- **the Main Theorem** (paper §1.2): $Syl_2\, K(Q_n) \oplus \mathbb{Z}_2 \cong \oplus_\lambda X(\lambda, (n-\lambda)/2)^{m_\lambda(n)}$, with the family groups $X(\lambda, i)$ given by the explicit rule of §1.2 (binary digits of $\lambda + 1$, raw clocks, integer antitonic fit) and the multiplicities $m_\lambda(n)$ given by the recursion of §1.2;
- **Theorem DW, the fold law** (§1.3): $Syl_2\, \bar{K}(n) \cong 2 \cdot Syl_2\, K(Q_n)$, where $\bar{K}(n)$ is the sandpile group of the folded cube $Q_n/\langle \mathbb{1} \rangle$, and, for $n \geq 2$, its equivalent form $Syl_2\, K(Q_n) \cong (\mathbb{Z}/2)^{a_n} \oplus H$ with H free of factors $\mathbb{Z}/2$ and $Syl_2\, \bar{K}(n) \cong 2H$;
- **Corollaries 1.4–1.7**: the odd part of $\bar{K}(n)$ (with the two statements above, the whole sandpile group of the folded cube); Bai's two counts; the $n + 1$ largest cyclic factors of $Syl_2\, K(Q_n)$ in all three cases, including Gao, Marx-Kuo, McDonald and Yuen's Theorem 4.1 for $n \geq 3$ and Theorem 4.2, their Conjecture 4.14 and the $(n+1)$-th factor of the 2019 poster; and their Conjecture 5.4;
- **Bai's odd part of $K(Q_n)$** (§1.1, [Bai03, Theorem 1.2]): for every odd prime p, $Syl_p\, K(Q_n) \cong Syl_p \oplus_{j=1}^{n} (\mathbb{Z}/j)^{C(n,j)}$;

and every one of the 68 numbered statements of §2–§10 on which these rest, from Lemma 2.1 to Theorem 10.22, Theorem D among them (§3).

The proof contains no `sorry`, no `admit`, no `native_decide` and no added axiom. Every theorem of the project depends only on the three standard axioms of Lean and Mathlib: `propext`, `Classical.choice` and `Quot.sound` (§4). **All 416 modules of the project were compiled again from their sources on the author's Mac, with exit code 0, in a clean rebuild that skipped none** (§8).

Nothing is cited. Unlike the Lean certificate of the Chaise Longue, whose Main Theorem rests on a topological result that Lean does not have, the statements here are about a finite graph: the Laplacian of Q_n, its cokernel and the torsion of that cokernel. Lean certifies the theorems of the paper themselves, not an algebraic form of them. The one place where trust in the author remains is the faithfulness of the statements: that the

Lean definitions say what the paper says. §1 prints every definition the statements use, verbatim, and compares it with the paper line by line.

Read cold. A reader other than its writer, with the compiled project at hand, read this certificate before release; verdict «holds as far as I can check», with four errors in the record (one sentence and three counts) and no error in a statement. They are corrected here (§2 item 7).

§1 states exactly what is proved. §2 states exactly what is not. §3–§8 describe how the result was obtained and how to check it.

---

1. The certified statements

1.1 The theorems, verbatim

Printed by Lean with `#check` (log `logs/CertCubo.log`, from `checks/CertCubo.lean`, run on the rebuilt project of §8). **Twenty-four statements are printed here.** Sixteen of them are the *headline theorems*, whose dependency cones are counted in §3.3: `M1`, `M2`, `theorem_D_i`, `BaiCounts.B1`, `B2`, `G1`, `G2`, `G3`, `Y2`, `FoldLaw.H`, `DW`, `DW'`, `L1`, `FoldLaw.R`, `OddParts.B1`, `C5`. With `MainTheorem.G2` and the four `M3_*` they are the twenty-one statements restated by hand in `CheckCubo.lean` (§1.4); with the three cases `P1`–`P3` of Corollary 1.6, the twenty-four printed here. The Main Theorem, in its multiplicity form and in its list form, with the identification of the group of a family and the support of the multiplicities:

```
MainTheorem.M1 : ∀ (n : ℕ),
  1 ≤ n →
    Nonempty
      (ℤ_[2] × ↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n)) ≃+
        ((q : (lam : Fin (n + 1)) × Fin (FamDec.m (↑lam) n)) → MainTheorem.XFam (↑q.fst) ((n - ↑q.fst)
/ 2)))
MainTheorem.M2 : ∀ (n : ℕ),
  1 ≤ n →
    Nonempty
      (ℤ_[2] × ↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n)) ≃+
        ((j : Fin (FamDec.L n).length) → MainTheorem.XFam ((FamDec.L n).get j) ((n - (FamDec.L n).get
j) / 2)))
MainTheorem.G2 : ∀ (lam : ℕ),
  1 ≤ lam → ∀ (i : ℕ), Nonempty (FamilyGroup.X (Families.ds lam) 1 i ≃ₗ[ℤ_[2]] MainTheorem.XFam lam i)
MainTheorem.M3_support : ∀ (n : ℕ), 1 ≤ n → ∀ (lam : ℕ), FamDec.m lam n ≠ 0 → 1 ≤ lam ∧ lam ≤ n ∧ lam
% 2 = n % 2
MainTheorem.M3_shift : ∀ (n : ℕ), 1 ≤ n → ∀ (lam : ℕ), FamDec.m lam n ≠ 0 → ((n - lam) / 2 = 0 ↔ lam =
n)
MainTheorem.M3_self : ∀ (n : ℕ), 1 ≤ n → FamDec.m n n = 1
MainTheorem.M3_zero : ∀ (n : ℕ), 1 ≤ n → FamDec.m 0 n = 0
```

Theorem DW, its equivalent form, the number of cyclic factors of $Syl_2\ \bar{K}(n)$ and the $\mathbb{F}_2$-rank of the folded Laplacian (§1.3, §10.8), and the fold law on every family (Corollary 10.17 and Theorem 10.22, assembled):

```
FoldLaw.DW : ∀ (n : ℕ),
  1 ≤ n →
    Nonempty (↥(CubeLaplacian.Syl2 ↥(Fold.KBar n)) ≃+ ↥(FoldGroup.twoM ↥(CubeLaplacian.Syl2
↥(CubeLaplacian.K n))))
FoldLaw.DW' : ∀ (n : ℕ),
  2 ≤ n →
    ∃ H x,
      ∃ (_ : Finite H),
        Nonempty (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n)) ≃+ (Fin (BaiCounts.aN n) → ZMod 2) × H) ∧
          Nonempty (↥(CubeLaplacian.Syl2 ↥(Fold.KBar n)) ≃+ ↥(FoldGroup.twoM H)) ∧ BaiCounts.l0 H =
BaiCounts.l1 H
FoldLaw.L1 : ∀ (n : ℕ),
  2 ≤ n → BaiCounts.l0 ↥(CubeLaplacian.Syl2 ↥(Fold.KBar n)) = 2 ^ (2 ^ (n - 1) - 1 - BaiCounts.aN n)
```

```
FoldLaw.R : ∀ (n : ℕ), 2 ≤ n → Module.finrank (ZMod 2) ↥(Fold.lapBar (ZMod 2) n).range = BaiCounts.aN
n
FoldLaw.H : ∀ (lam : ℕ),
  1 ≤ lam → ∀ (i : ℕ), Nonempty (FoldGroup.XbarFam lam i ≃+ ↥(FoldGroup.twoM (FamilyGroup.X
(Families.ds lam) 1 i)))
```

In `DW'` the binder *x* printed by Lean is the group structure on *H*: the statement is `∃ (H : Type) (_ : AddCommGroup H) (_ : Finite H), …`, as `CheckCubo.lean` restates it by hand.

Corollary 1.5 (Bai's counts), Corollary 1.6 (its three cases `P1`–`P3`, and the four statements of «In particular» `G1`–`G3`), Corollary 1.7:

```
BaiCounts.B1 : ∀ (n : ℕ), 1 ≤ n → BaiCounts.l0 ↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n)) = 2 ^ (2 ^
(n - 1) - 1)
BaiCounts.B2 : ∀ (n : ℕ),
  2 ≤ n → BaiCounts.l1 ↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n)) = 2 ^ (2 ^ (n - 1) - 1 -
BaiCounts.aN n)
PodiumCor.P1 : ∀ (n : ℕ),
  4 ≤ n →
    n % 2 = 0 →
      ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) 1) =
          max (PodiumCor.gmax (n - 1)) (Rulers.phi n - 1) ∧
        ∀ (r : ℕ),
          2 ≤ r → r ≤ n + 1 → ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) r) =
PodiumCor.gmax (n - 1)
PodiumCor.P2 : ∀ (n : ℕ),
  4 ≤ n →
    n % 2 = 1 →
      Rulers.phi (n - 1) ≤ PodiumCor.gmax (n - 1) →
        ∀ (r : ℕ),
          1 ≤ r → r ≤ n + 1 → ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) r) =
PodiumCor.gmax (n - 1)
PodiumCor.P3 : ∀ (n : ℕ),
  4 ≤ n →
    n % 2 = 1 →
      PodiumCor.gmax (n - 1) < Rulers.phi (n - 1) →
        (∀ (r : ℕ),
            1 ≤ r → r ≤ n - 1 → ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) r) =
Rulers.phi (n - 1)) ∧
          ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) n) =
              max (PodiumCor.gmax (n - 1)) (Rulers.phi (n - 1) - 2) ∧
            ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) (n + 1)) = PodiumCor.gmax (n
- 1)
PodiumCor.G1 : ∀ (n : ℕ),
  3 ≤ n →
    ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) 1) =
        max (PodiumCor.gmax n) (↑(Rulers.v2 n) + ↑n - 1) ∧
      ∀ (r : ℕ), 2 ≤ r → r ≤ n - 1 → ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) r)
= PodiumCor.gmax n
PodiumCor.G2 : ∀ (n : ℕ),
  3 ≤ n →
    ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) n) =
      max (PodiumCor.gmax (n - 1)) (↑(Rulers.v2 (n - 1)) + ↑n - 3)
PodiumCor.G3 : ∀ (n : ℕ),
  4 ≤ n → ↑(PodiumCor.cExp (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K n))) (n + 1)) = PodiumCor.gmax (n -
1)
Mirror.Y2 : ∀ (e : ℕ),
  1 ≤ e →
    Nonempty
      (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K (2 ^ e))) ≃+
        (↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K (2 ^ e - 1))) × ↥(CubeLaplacian.Syl2 ↥(CubeLaplacian.K
(2 ^ e - 1)))) ×
          ZMod (2 ^ (2 ^ e + e - 1)))
```

The odd parts: Bai's odd part of $K(Q_n)$, and the odd part of $\bar{K}(n)$ (Corollary 1.4, second sentence):

```
OddParts.B1 : ∀ (n p : ℕ),
  Nat.Prime p →
```

```
    p ≠ 2 →
      Nonempty
        (ɪ(OddParts.Sylp p ɪ(CubeLaplacian.K n)) ≃+
          ɪ(OddParts.Sylp p ((j : Fin n) → Fin (n.choose (↑j + 1)) → ZMod (↑j + 1))))
OddParts.C5 : ∀ (n : ℕ),
  1 ≤ n →
    ∀ (p : ℕ),
      Nat.Prime p →
        p ≠ 2 →
          Nonempty
            (ɪ(OddParts.Sylp p ɪ(Fold.KBar n)) ≃+
              ɪ(OddParts.Sylp p ((t : Fin (n / 2)) → Fin (n.choose (2 * (↑t + 1))) → ZMod (2 * (2 *
(↑t + 1))))))
```

Theorem D (§4), over $\mathbb{Z}$, for an arbitrary digit list `ds` and every $\alpha \geq 1$. Its final matrix `Windows.finalMatrix` is the one of Theorem 4.4; it is on the path of the Main Theorem and is not part of the Main Theorem's statement, so a reader who checks only the Main Theorem need not read it:

```
Windows.theorem_D_i : ∀ (ds : List Bool) (α i : ℕ),
  1 ≤ α →
    Nonempty
      ((Dance.SysT ℤ ds (Fin 1) ⧸ (Dance.sigma ℤ ds α i).range) ≃ₗ[ℤ]
        ((a : Fin ds.length) → (Fin (2 ^ (ds.length + Dance.mCount ds - 1 - ↑a)) → ℤ) ⧸ 2 ^ ↑a • ⊤) ×
          (Fin (2 ^ Dance.mCount ds) → ℤ) ⧸ (2 ^ ds.length • Matrix.toLin' (Windows.finalMatrix ℤ ds α
i)).range)
```

1.2 The definitions the statements use, verbatim

Printed by `#print` in the same log. They are grouped as in the paper. A reader who wants to check the Main Theorem and Theorem DW against the paper needs these definitions and nothing else.

The cube, its sandpile group, and the *p*-parts.

```
@[reducible] def CubeLaplacian.G : ℕ → Type :=
fun n => Fin n → ZMod 2
def CubeLaplacian.e : (n : ℕ) → Fin n → CubeLaplacian.G n :=
fun n i => Pi.single i 1
def CubeLaplacian.lap.{u_1} : (A : Type u_1) →
  [inst : CommRing A] → (n : ℕ) → (CubeLaplacian.G n → A) →ₗ[A] CubeLaplacian.G n → A :=
fun A [CommRing A] n =>
  { toFun := fun f g => ↑n * f g - ∑ i, f (g + CubeLaplacian.e n i), map_add' := …, map_smul' := … }
@[reducible] def CubeLaplacian.coker : ℕ → Type :=
fun n => (CubeLaplacian.G n → ℤ) ⧸ (CubeLaplacian.L n).range
@[reducible] def CubeLaplacian.K : (n : ℕ) → Submodule ℤ (CubeLaplacian.coker n) :=
fun n => Submodule.torsion ℤ (CubeLaplacian.coker n)
def CubeLaplacian.Syl2.{u_1} : (T : Type u_1) → [inst : AddCommGroup T] → AddSubgroup T :=
fun T [AddCommGroup T] => { carrier := {y | ∃ k, 2 ^ k • y = 0}, add_mem' := …, zero_mem' := …,
neg_mem' := … }
def OddParts.Sylp.{u_1} : ℕ → (T : Type u_1) → [inst : AddCommGroup T] → AddSubgroup T :=
fun p T [AddCommGroup T] => { carrier := {y | ∃ k, p ^ k • y = 0}, add_mem' := …, zero_mem' := …,
neg_mem' := … }
```

Here `CubeLaplacian.L n` is `lap ℤ n` (an `abbrev`). `checks/CheckCubo.lean` restates the cokernel by hand as `(G n → ℤ) ⧸ LinearMap.range (lap ℤ n)` and Lean accepts it by `rfl`.

The folded cube.

```
def Fold.aElt : (n : ℕ) → CubeLaplacian.G n :=
fun n x => 1
@[reducible] def Fold.Gbar : ℕ → Type :=
fun n => CubeLaplacian.G n ⧸ AddSubgroup.zmultiples (Fold.aElt n)
def Fold.mk : (n : ℕ) → CubeLaplacian.G n →+ Fold.Gbar n :=
fun n => QuotientAddGroup.mk' (AddSubgroup.zmultiples (Fold.aElt n))
def Fold.lapBar.{u_1} : (A : Type u_1) → [inst : CommRing A] → (n : ℕ) → (Fold.Gbar n → A) →ₗ[A]
Fold.Gbar n → A :=
```

```
fun A [CommRing A] n =>
  { toFun := fun u x => ↑n * u x - ∑ i, u (x + (Fold.mk n) (CubeLaplacian.e n i)), map_add' := …,
map_smul' := … }
@[reducible] def Fold.KBar : (n : ℕ) → Submodule ℤ (Fold.cokerBar n) :=
fun n => Submodule.torsion ℤ (Fold.cokerBar n)
def FoldGroup.twoM.{u_1} : (M : Type u_1) → [inst : AddCommGroup M] → AddSubgroup M :=
fun M [AddCommGroup M] => (nsmulAddMonoidHom 2).range
```

Here `Fold.cokerBar n` is `(Gbar n → ℤ) ⧸ LinearMap.range (lapBar ℤ n)`; `CheckCubo.lean` restates `KBar n` by hand as `Submodule.torsion ℤ ((Gbar n → ℤ) ⧸ LinearMap.range (lapBar ℤ n))` and Lean accepts it by `rfl`.

The multiplicities.

```
@[irreducible] def FamDec.Vdot : ℕ → List ℕ :=
WellFounded.Nat.fix (fun x => x) fun a a_1 =>
  (match (motive := (x : ℕ) → ((y : ℕ) → InvImage (fun x1 x2 => x1 < x2) (fun x => x) y x → List ℕ) →
List ℕ) a with
    | 0 => fun x => [1]
    | n.succ => fun x => if h : n % 2 = 0 then [n + 2] else [n, n] ++ List.map (fun ν => 2 * ν + 1) (x
(n / 2) …))
    a_1
def FamDec.L : ℕ → List ℕ :=
fun x =>
  Nat.brecOn x fun x f =>
    (match (motive := (x : ℕ) → Nat.below x → List ℕ) x with
      | 0 => fun x => [0]
      | n.succ => fun x => List.flatMap FamDec.Vdot x.1)
      f
def FamDec.m : ℕ → ℕ → ℕ :=
fun lam n => List.count lam (FamDec.L n)
```

The family of λ and its dancers.

```
@[irreducible] def Families.ds : ℕ → List Bool :=
WellFounded.Nat.fix (fun x => x) fun a a_1 =>
  (match (motive := (x : ℕ) → ((y : ℕ) → InvImage (fun x1 x2 => x1 < x2) (fun x => x) y x → List Bool)
→ List Bool)
      a with
    | 0 => fun x => []
    | n.succ => fun x => if h : n % 2 = 0 then false :: x (n / 2) … else true :: x (n / 2) …)
    a_1
def Windows.Ids : List Bool → Finset ℕ :=
fun ds => {t ∈ Finset.range ds.length | ds[t]? = some true}
def Dance.mCount : List Bool → ℕ :=
fun x =>
  List.brecOn x fun x f =>
    (match (motive := (x : List Bool) → List.below x → ℕ) x with
      | [] => fun x => 0
      | false :: rest => fun x => x.1
      | true :: rest => fun x => x.1 + 1)
      f
def TheoremF.dancer : Finset ℕ → ℕ → Finset ℕ :=
fun I p => {t ∈ I | Houses.bit {x ∈ I | x < t}.card p = 1}
def Houses.bit : ℕ → ℕ → ℕ :=
fun u x => x / 2 ^ u % 2
```

The raw clocks.

```
def Rulers.s2 : ℕ → ℕ :=
fun x => List.count 1 (Nat.digits 2 x)
def Rulers.v2 : ℕ → ℕ :=
fun x => padicValNat 2 x
def Rulers.kappa : ℕ → ℕ → ℤ :=
fun a b => ↑(Rulers.s2 a) + ↑(Rulers.s2 b) - ↑(Rulers.s2 (a + b))
def Rulers.phi : ℕ → ℤ :=
fun x => ↑x + ↑(Rulers.v2 x)
```

```
def Rulers.oE : Finset ℕ → ℕ :=
fun E => ∑ t ∈ E, 2 ^ t
def Rulers.minOr : ℕ → Finset ℕ → ℕ :=
fun k E => if h : E.Nonempty then E.min' h else k
def Rulers.next : ℕ → Finset ℕ → ℕ → ℕ :=
fun k I t => Rulers.minOr k ({u ∈ insert k I | t < u})
def Rulers.h : ℕ → Finset ℕ → ℕ → ℕ :=
fun k I t => Rulers.next k I t - t
def Rulers.hSum : ℕ → Finset ℕ → Finset ℕ → ℕ :=
fun k I E => ∑ t ∈ E, Rulers.h k I t
def Rulers.lam : ℕ → Finset ℕ → ℕ :=
fun k I => 2 ^ k + Rulers.oE I - 1
def Rulers.xi : ℕ → Finset ℕ → Finset ℕ → ℕ :=
fun k I D => Rulers.lam k I + 1 - 2 * Rulers.oE D
def Rulers.clock : ℕ → Finset ℕ → ℕ → Finset ℕ → ℤ :=
fun k I i D =>
  Rulers.phi (Rulers.xi k I D) + Rulers.kappa (i + Rulers.oE D - 1) (Rulers.xi k I D) + ↑(Rulers.hSum
k I D)
```

The antitonic fit and the integer fit. The term `….choose` in `fit` is the choice of a minimizer from the existence statement `AntitonicFit.exists_isFit`; uniqueness is `AntitonicFit.existsUnique_fit`.

```
def AntitonicFit.NonIncr : ℕ → (ℕ → ℝ) → Prop :=
fun N z => ∀ (i : ℕ), i + 1 < N → z (i + 1) ≤ z i
def AntitonicFit.sqDist : ℕ → (ℕ → ℝ) → (ℕ → ℝ) → ℝ :=
fun N x z => ∑ j ∈ Finset.range N, (x j - z j) ^ 2
def AntitonicFit.IsFit : ℕ → (ℕ → ℝ) → (ℕ → ℝ) → Prop :=
fun N x z =>
  AntitonicFit.NonIncr N z ∧
    ∀ (y : ℕ → ℝ), AntitonicFit.NonIncr N y → AntitonicFit.sqDist N x z ≤ AntitonicFit.sqDist N x y
def AntitonicFit.fit : ℕ → (ℕ → ℝ) → ℕ → ℝ :=
fun N x j => if j < N then ….choose j else 0
def AntitonicFit.levelSet : ℕ → (ℕ → ℝ) → ℕ → Finset ℕ :=
fun N z j => {i ∈ Finset.range N | z i = z j}
def AntitonicFit.toR : (ℕ → ℤ) → ℕ → ℝ :=
fun x j => ↑(x j)
def AntitonicFit.blockF : ℤ → ℕ → ℕ → ℤ :=
fun S P k => if ↑k < S % ↑P then ⌈↑S / ↑P⌉ else ⌊↑S / ↑P⌋
def AntitonicFit.ifit : ℕ → (ℕ → ℤ) → ℕ → ℤ :=
fun N x j =>
  if j < N then
    AntitonicFit.blockF (∑ i ∈ AntitonicFit.levelSet N (AntitonicFit.fit N (AntitonicFit.toR x)) j, x
i)
      (AntitonicFit.levelSet N (AntitonicFit.fit N (AntitonicFit.toR x)) j).card
      {x ∈ AntitonicFit.levelSet N (AntitonicFit.fit N (AntitonicFit.toR x)) j | x < j}.card
  else 0
```

The group of a family.

```
@[reducible] def FamilyGroup.SC : List Bool → Type :=
fun ds => (a : Fin ds.length) → (Fin (2 ^ (ds.length + Dance.mCount ds - 1 - ↑a)) → ℤ_[2]) ⧸ 2 ^ ↑a •
⊤
@[reducible] def FamilyGroup.Q : ℕ → Type :=
fun e => ℤ_[2] ⧸ Ideal.span {2 ^ e}
def FamilyGroup.yPool : List Bool → ℕ → ℕ → ℤ :=
fun ds i =>
  AntitonicFit.ifit (2 ^ Dance.mCount ds) fun j =>
    Rulers.clock ds.length (Windows.Ids ds) i (TheoremF.dancer (Windows.Ids ds) j)
def FamilyGroup.yPool0 : List Bool → ℕ → ℤ :=
fun ds =>
  AntitonicFit.ifit (2 ^ Dance.mCount ds - 1) fun j =>
    Rulers.clock ds.length (Windows.Ids ds) 0 (TheoremF.dancer (Windows.Ids ds) (j + 1))
@[reducible] def MainTheorem.XFam : ℕ → ℕ → Type :=
fun lam x =>
  match x with
  | 0 =>
    FamilyGroup.SC (Families.ds lam) ×
      ℤ_[2] ×
        ((q : Fin (2 ^ Dance.mCount (Families.ds lam) - 1)) →
```

```
          FamilyGroup.Q (FamilyGroup.yPool0 (Families.ds lam) ↑q).toNat)
  | i.succ =>
    FamilyGroup.SC (Families.ds lam) ×
      ((p : Fin (2 ^ Dance.mCount (Families.ds lam))) →
        FamilyGroup.Q (FamilyGroup.yPool (Families.ds lam) (i + 1) ↑p).toNat)
```

The counts and the exponents.

```
def BaiCounts.aN : ℕ → ℕ :=
fun n => 2 ^ (n - 2) - 2 ^ ((n - 2) / 2)
def BaiCounts.twoPow.{u_1} : (G : Type u_1) → [inst : AddCommGroup G] → ℕ → AddSubgroup G :=
fun G [AddCommGroup G] j => { carrier := {x | ∃ y, 2 ^ j • y = x}, add_mem' := …, zero_mem' := …,
neg_mem' := … }
def BaiCounts.l0.{u_1} : (G : Type u_1) → [AddCommGroup G] → ℕ :=
fun G [AddCommGroup G] => Nat.card (G ⧸ BaiCounts.twoPow G 1)
def BaiCounts.l1.{u_1} : (G : Type u_1) → [AddCommGroup G] → ℕ :=
fun G [AddCommGroup G] =>
  Nat.card (↥(BaiCounts.twoPow G 1) ⧸ (BaiCounts.twoPow G 2).addSubgroupOf (BaiCounts.twoPow G 1))
def PodiumCor.layer.{u_1} : (G : Type u_1) → [AddCommGroup G] → ℕ → ℕ :=
fun G [AddCommGroup G] j =>
  Nat.card (↥(BaiCounts.twoPow G j) ⧸ (BaiCounts.twoPow G (j + 1)).addSubgroupOf (BaiCounts.twoPow G
j))
def PodiumCor.cExp.{u_1} : (G : Type u_1) → [AddCommGroup G] → ℕ → ℕ :=
fun G [AddCommGroup G] r => sSup {e | 1 ≤ e ∧ 2 ^ r ≤ PodiumCor.layer G (e - 1)}
def PodiumCor.gmax : ℕ → ℤ :=
fun N => if h : 1 < N then (Finset.Ico 1 N).sup' … Rulers.phi else 0
```

1.3 Faithfulness to the paper, line by line

The middle column names the Lean definitions or theorems; their full text is printed in §1.1–§1.2.

Paper (v8)	Lean	Agreement
Q_n: vertices $\{0,1\}^n$, edges between words that differ in one letter (§1.1)	`G`, `e`	Literal: `G n = Fin n → ZMod 2`, and the neighbours of g are the `g + e i`, with `e i = Pi.single i 1`.
the Laplacian $L(Q_n)$ acting on $\mathbb{Z}^V$	`lap`, `CubeLaplacian.L`	Literal: `lap ℤ n f g = n·f g − Σ_i f (g + e i)`, degree n minus the adjacency; `CubeLaplacian.L n = lap ℤ n`.
$K(Q_n)$: the finite group with $coker\, L(Q_n) = \mathbb{Z} \oplus K(Q_n)$ (§1.1)	`coker`, `K`	`K n` is the torsion subgroup of `coker n`. Lean proves `coker n ≃ ℤ × K n` (`CubeLaplacian.cokerEquivK`, piece C17), so the two definitions agree.
$Syl_2 A$, $Syl_p A$	`Syl2`, `Sylp`	Literal: the elements killed by a power of 2, resp. of p.
the folded cube $Q_n/\langle \mathbb{1} \rangle$: vertices the 2^{n-1} pairs $\{x, x+\mathbb{1}\}$ of antipodal words; $\{x, x+\mathbb{1}\}$ is joined to $\{x+e_i, x+\mathbb{1}+e_i\}$ once for each $i = 1, \ldots, n$; the Cayley graph of $\mathbb{F}_2^n/\langle \mathbb{1} \rangle$ with the images of the unit vectors (§1.3, in the wording of version 9; see the third column)	`aElt`, `Gbar`, `mk`, `lapBar`	`Gbar n` is `G n` modulo `aElt n = 𝟙`, and `lapBar ℤ n u x = n·u x − Σ_i u (x + mk (e i))`: the Laplacian of the Cayley multigraph of $\mathbb{F}_2^n/\langle \mathbb{1} \rangle$ with the n images of the unit vectors, one edge for each i. At $n = 2$ the two images of e_1, e_2 coincide and give the double edge of §1.3. Version 8 of the paper describes the edges as «once for each pair of words, one from each, that differ in one letter»; read literally, that sentence counts each edge twice ($(x, x+e_i)$ and $(x+\mathbb{1}, x+\mathbb{1}+e_i)$). The paper's next sentence, its proofs and its numbers use the Cayley graph, as Lean does.
$\bar{K}(n)$	`KBar`	As for `K`: the torsion of `(Gbar n → ℤ) ⧸ range (lapBar ℤ n)`. Lean proves `cokerBar n ≃ ℤ × KBar n` and that `KBar n` is finite (`FoldGroup.cokerBarEquivKBar`, `FoldGroup.KBar_finite`, C38b).

Paper (v8)	Lean	Agreement
$2{\cdot}A$, «the subgroup of doubles»	`twoM`	Literal: the range of $x \mapsto 2x$.
the multiplicities: $V{\cdot}[0] = [1]$, $V{\cdot}[2l+1] = [2l+2]$, $V{\cdot}[2l+2] = 2[2l+1] + \Phi(V{\cdot}[l])$, $\Phi[\nu] = [2\nu+1]$, and $\Sigma\, m_\lambda(n)[\lambda] = V{\cdot}\Sigma\, m_\lambda(n-1)[\lambda]$ from $[0]$ (§1.2)	`Vdot`, `FamDec.L`, `FamDec.m`	A formal sum is a list of its symbols with repetition. `Vdot 0 = [1]`; for $n+1 = 2l+1$ (n even) `Vdot` gives $[2l+2]$; for $n+1 = 2l+2$ (n odd) it gives $[2l+1, 2l+1]$ followed by $\Phi(V{\cdot}[l])$, since $n/2 = l$. Lean proves these three equations (`FamDec.Vdot_zero`, `Vdot_odd`, `Vdot_even`). `L (n+1) = (L n).flatMap Vdot` is the additive extension of $V{\cdot}$, and `m λ n = (L n).count λ`.
$\lambda + 1 = 2^k + \Sigma_{t\in I}\, 2^t$, $I \subseteq \{0, \ldots, k-1\}$ (§1.2)	`ds`, `Ids`, `mCount`	`ds λ` is the list of the binary digits of $\lambda + 1$ below the leading one, lowest first: `ds 0 = []`, `ds (2l+1) = false :: ds l` ($\lambda + 1 = 2(l+1)$, lowest digit 0), `ds (2l+2) = true :: ds l` ($\lambda + 1 = 2(l+1) + 1$, lowest digit 1). Then `k = (ds λ).length` and `I = Ids (ds λ)`, the positions of `true`. Lean proves $\lambda + 1 = 2^k + \Sigma_{t\in I}\, 2^t$ (`Families.B0b`, C32) and `|I| = mCount (ds λ)` (`Windows.card_Ids`).
$next(t) = min\{u \in I \cup \{k\} : u > t\}$, $h_t = next(t) - t$, $h(D) = \Sigma_{t\in D}\, h_t$, $o_D = \Sigma_{t\in D}\, 2^t$	`next`, `h`, `hSum`, `oE`, `minOr`	Literal. For $t \in I$ the set contains k, so the convention `min ∅ := k` of `minOr` is never used.
$\kappa(a, b) = s_2(a) + s_2(b) - s_2(a+b)$, $\varphi(x) = x + v_2(x)$	`kappa`, `phi`, `s2`, `v2`	Literal: `s2 x` counts the ones of `Nat.digits 2 x`, and `v2 = padicValNat 2`.
$\xi_D = \lambda + 1 - 2o_D$; raw clock $u_i(D) = \varphi(\xi_D) + \kappa(i + o_D - 1, \xi_D) + h(D)$ for $(i, D) \neq (0, \varnothing)$	`lam`, `xi`, `clock`	Literal, with `lam k I = 2^k + oE I - 1`. The subtractions are of natural numbers; Lean proves that $\xi_D \geq 1$ and that the subtraction does not truncate (`Rulers.lemma_5_0_xi`, C3). $i + o_D - 1$ does not truncate when $(i, D) \neq (0, \varnothing)$, the only case where `clock` is used.
the o-order of the dancers, from $\varnothing$ to I	`dancer`, `bit`	`dancer I p` is the set of $t \in I$ whose rank in I is a binary digit 1 of p. Lean proves that $p \mapsto o_{dancer\ I\ p}$ is strictly increasing on $p < 2^{\|I\|}$ (`TheoremF.lemma_0b`), that every $D \subseteq I$ is a dancer (`TheoremF.lemma_0c_existsUnique`), and that `dancer I 0 = ∅`. So `dancer I p` is the p-th dancer in o-order.
the antitonic fit: the non-increasing sequence nearest to x in the Euclidean norm; its level sets; the integer fit: on a level set of p positions with sum S, $S \bmod p$ copies of $\lfloor S/p \rfloor$ followed by $p - (S \bmod p)$ copies of $\lfloor S/p \rfloor$ (§1.2, §7.1)	`NonIncr`, `sqDist`, `IsFit`, `fit`, `levelSet`, `blockF`, `ifit`	`fit N x` is a minimizer of `sqDist` among the non-increasing sequences, set to 0 from position N on; it is unique (`AntitonicFit.existsUnique_fit`). `levelSet N z j` is the set of positions with the value of j. `ifit` applies `blockF` on each level set: $\lfloor S/P \rfloor$ at the positions of rank `r < S % P`, $\lfloor S/P \rfloor$ after. `S % P` is the non-negative remainder for $P > 0$, the paper's $S \bmod p$. For a non-increasing z a level set is an interval, and the rank of j in $[p, q]$ is $j - p$ (both proved in C1), so the ceilings come first. The paper also describes how to compute the fit, by the pool-adjacent-violators algorithm; Lean does not formalize the algorithm, only the fit it computes (§2 item 3).
$\hat{y}_i$: the integer antitonic fit of the raw clocks $u_i(D)$, read in o-order; at shift 0 the entry ∞ of $\varnothing$ stands first and is not pooled (§1.2)	`yPool`, `yPool0`	Literal: `yPool ds i` is the integer fit of the clocks of the $2^{\|I\|}$ dancers in o-order; `yPool0 ds` is the integer fit of the clocks of the $2^{\|I\|} - 1$ dancers after $\varnothing$.

Paper (v8)	Lean	Agreement
$X(\lambda,\, i) = \bigoplus_{a=1}^{k-1} (\mathbb{Z}/2^a)^{2^{\lvert I\rvert+k-1-a}} \oplus \bigoplus_{D\subseteq I} \mathbb{Z}/2^{\hat{y}_i(D)}$, with $\mathbb{Z}/2\text{^}\infty$:= $\mathbb{Z}_2$	`XFam`, `SC`, `Q`	`SC` is the product over $a < k$ of $(\mathbb{Z}_2)^{2^{k+\lvert I\rvert-1-a}}$ modulo 2^a; its extra index $a = 0$ gives the factor $\mathbb{Z}_2{}^r \,/\, 2^0 = 0$, and $\mathbb{Z}_2 \,/\, (2^a) \cong \mathbb{Z}/2^a$. $Q\ e = \mathbb{Z}_2 \,/\, (2^e)$. At shift $i \geq 1$, `XFam` is `SC` times the big clocks $Q(\hat{y}_i(p))$, $p < 2^{\lvert I\rvert}$; at shift 0, `SC` times $\mathbb{Z}_2$ (the big clock $\hat{y}_0(\varnothing) = \infty$) times the $Q(\hat{y}_0(q))$ of the other dancers. The exponents are integers turned into natural numbers by `toNat`; Lean proves they are ≥ 0 (`MainTheorem.G1_pos`, `G1_zero`), so `toNat` changes nothing.
the Main Theorem: for every $n \geq 1$, as $\mathbb{Z}_2$-modules, $Syl_2\, K(Q_n) \oplus \mathbb{Z}_2 \cong \bigoplus_{1\leq\lambda\leq n,\ \lambda\equiv n\ (2)} X(\lambda,\, (n-\lambda)/2)^{m_\lambda(n)}$	`M1`, `M2`, `M3_support`	(i) `M1` takes the product over all $\lambda \leq n$, with `m λ n` copies of `XFam λ ((n − λ)/2)`; `M3_support` shows `m λ n = 0` unless $1 \leq \lambda \leq n$ and $\lambda \equiv n\ (mod\ 2)$, so it is the paper's sum. (ii) A finite product is the direct sum. (iii) $\simeq+$ is an isomorphism of groups. It is automatically $\mathbb{Z}_2$-linear, because both sides are finitely generated $\mathbb{Z}_2$-modules (the finite 2-group $Syl_2\, K(Q_n)$ in a unique way), hence 2-adically separated and complete: an additive map f satisfies $f(2^j x) = 2^j f(x)$, so it is continuous for the 2-adic topologies; then $f(ax) = a\, f(x)$ holds for $a \in \mathbb{Z}$, and by density for $a \in \mathbb{Z}_2$. So the group isomorphism of Lean and the module isomorphism of the paper are equivalent statements. `M2` is the same with the list `L n` in place of the multiplicities.
Theorem DW and its equivalent form (§1.3)	`DW`, `DW'`	`DW` is literal, for $n \geq 1$. The paper's equivalent form says $Syl_2\, K(Q_n) \cong (\mathbb{Z}/2)^{a_n} \oplus$ *($Syl_2\, \bar{K}(n)$ with every exponent raised by one)*. `DW'` gives, for $n \geq 2$, a finite H with $Syl_2\, K(Q_n) \cong (\mathbb{Z}/2)^{a_n} \times H$, $Syl_2\, \bar{K}(n) \cong 2H$ and `l0 H = l1 H`, that is, $\lvert H/2H\rvert = \lvert 2H/4H\rvert$: every cyclic factor of H has order ≥ 4. Such an H is $Syl_2\, \bar{K}(n)$ with every exponent raised by one. H is a 2-group because it is a direct factor of one. `DW'` is stated for $n \geq 2$. At $n = 1$ the paper's $a_1 = 2^{-1} - 2^{-1} = 0$ and both groups are trivial (`B1` at $n = 1$ gives `l0 (Syl2 (K 1)) = 1`), but Lean does not state that case.
$a_n = 2^{n-2} - 2^{\lfloor (n-2)/2 \rfloor}$	`aN`	Literal, in $\mathbb{N}$; no truncation for $n \geq 2$.
Corollary 1.5: $2^{n-1} - 1$ cyclic factors, a_n of them $\mathbb{Z}/2$	`B1`, `B2`, `l0`, `l1`	$l0\ S = \lvert S/2S\rvert$ and $l1\ S = \lvert 2S/4S\rvert$. For a finite abelian 2-group, $\lvert S/2S\rvert$ is 2 to the number of cyclic factors and $\lvert 2S/4S\rvert$ is 2 to the number of factors of order ≥ 4. So `B1` ($l0 = 2^{2^{n-1}-1}$) and `B2` ($l1 = 2^{2^{n-1}-1-a_n}$) are equivalent to the two counts.
Corollary 1.6: c_r, the r-th largest exponent; $g(N) = max_{1\leq x<N}\, \varphi(x)$; $G = g(n-1)$	`cExp`, `layer`, `gmax`, `P1`–`P3`, `G1`–`G3`	$layer\ S\ j = \lvert 2^j\, S \,/\, 2^{j+1}\, S\rvert$ is 2 to the number of factors of order $> 2^j$, so `cExp S r`, the largest `e` with $2^r \leq layer\ S\ (e-1)$, is the largest `e` with at least r factors of exponent $\geq e$, which is c_r. Lean proves this reading (`PodiumCor.cExp_eq_getD`, `E2`, `E3`). `gmax` is literal. `P1`–`P3` are the three cases of the corollary for $n \geq 4$; `G1`–`G3` the four statements of «In particular», for $n \geq 3$, $n \geq 3$, $n \geq 4$. Gao et al. state their Theorem 4.1 for $n \geq 2$; the case $n = 2$ ($K(Q_2) = \mathbb{Z}/4$) is not in Lean.
Corollary 1.7: $Syl_2\, K(Q_{2^e}) \cong Syl_2\, K(Q_{2^e-1})^2 \oplus \mathbb{Z}/2^{2^e+e-1}$, $e \geq 1$	`Y2`	Literal.
Corollary 1.4: (2-part) $Syl_2\, \bar{K}(n)$ is the group of the Main Theorem with the a_n factors $\mathbb{Z}/2$ removed and every other exponent lowered by one; (odd part) $Syl_p\, \bar{K}(n) \cong Syl_p \bigoplus_{j\ even,\ 2\leq j\leq n} (\mathbb{Z}/2j)^{C(n,j)}$	2-part: `DW`, `DW'`, `M1`; odd part: `C5`	The odd part is literal, with $j = 2(t+1)$ for $t < n/2$. The 2-part is the reading of `DW'` given above, combined with `M1`; Lean has no separate theorem for it.
Bai's odd part (§1.1): $Syl_p\, K(Q_n) \cong Syl_p \bigoplus_{j=1}^{n} (\mathbb{Z}/j)^{C(n,j)}$	`OddParts.B1`	Literal, for every prime $p \neq 2$ and every n, with $j = j' + 1$ for $j' < n$.

Generality. Every statement holds for every n for which the paper states it, and `OddParts.B1` for every $n \geq 0$. The paper's results for which Lean asks for more than the paper are `DW'`, `L1` and `FoldLaw.R`, stated in Lean for $n \geq 2$; at $n = 1$ every group involved is trivial and every count is 0, but Lean does not state that case. In Corollary 1.6, Gao et al.'s Theorem 4.1 at $n = 2$ is not in Lean.

1.4 The final check file

`checks/CheckCubo.lean` restates **by hand**, with every type written out, the objects of §1.2 (by `rfl`) and the twenty-one statements of §1.1 named there (closed by the name of the certified theorem). Lean accepts every restatement. If a statement in the project differed from the restatement, the file would not compile. Then `#print axioms` for each of the twenty-one theorems; every line is `[propext, Classical.choice, Quot.sound]` (log `logs/CheckCubo_postclean.log`, exit code 0, 213 s, peak 1.56 GiB). The three cases `P1–P3` of Corollary 1.6 are printed in `CertCubo.lean`, with the same three axioms each.

---

2. What is not certified

1. **Nothing of the proof of the Main Theorem, of Theorem DW or of Corollaries 1.4–1.7 is cited or assumed.** Kirchhoff's theorem (the order of $K(G)$ is the number of spanning trees) is quoted in §1.1 of the paper as background and is not used.
2. **Faithfulness is an audit, not a theorem.** Lean guarantees that each proof is correct for the statement as written. That the Lean statements say what the paper says is checked in §1.3 by the author's audit. **A reader should check §1.2–§1.3 independently: about fifty short definitions, printed in §1.2.** This is the one place where trust in the author remains.
3. **The algorithm is not formalized; the fit is.** The paper computes the antitonic fit by the pool-adjacent-violators algorithm [ABERS55], [BBBB72]. Lean defines the fit as the least-squares projection, as §7.1 of the paper does. Lean proves the characterization of Lemma 7.1, from which the correctness of the algorithm follows, but it does not formalize the algorithm itself.
4. **Representation theory: the names, not the objects.**
   - The paper says that $m_\lambda(n)$ is the multiplicity of the indecomposable tilting module $T(\lambda)$ of SL_2 in characteristic 2 in $V^{\otimes n}$. Lean proves $V^{\otimes n} \cong \bigoplus_\lambda T(\lambda)^{m_\lambda(n)}$ over $\mathbb{Z}_2$ (Theorem 3.6, piece C31), where $T(\lambda)$ are the explicit dance lattices of §3.3.
   - That these lattices are the indecomposable tilting modules is not formalized. It is not used, and it does not occur in any statement.
   - The same holds for the reading of the hyperalgebra of §2.2: Lean works with the explicit divided-power lattices of the paper.
5. **Numerical facts of the paper are not certified as theorems:**
   - the examples ($n = 5$, $n = 6$) and the statement that the right-hand side takes seconds at $n = 1024$;
   - the sentence of §1.3 that the quotient by x_1x_2 does not halve the group («we checked $3 \leq n \leq 12$»), which is a measurement;
   - the computations of §12;
   - the remark of §1.2 that each copy of $X(\lambda, i)$ has $2^{|I|+k-1} = \dim T(\lambda)/2$ cyclic factors (Lean proves Bai's total count, Corollary 1.5, instead);
   - the literature and the vocabulary of §1.7–§1.8;
   - Remark (2) of §10.5, an observation used nowhere.

6. **Seven modules of the project are outside the cone of every headline theorem** (§3.3). They are examples, an earlier form of Corollary 10.17, and alternative forms of Lemmas 3.3, 3.5 and 7.6. They compile, and they are not needed.
7. **The cold reading.** On 11 October 2026 a reader other than the writer (a separate Claude session, with the paper, the sources, the pieces, the logs and the compiled project, but not the author's notes) read this certificate. It re-read every definition in the statements against the paper; printed the statements and their axioms itself; re-elaborated from source the eight files of `M1`, `DW`, `DW'` and five more of the chain; evaluated the multiplicities and 2 121 raw clocks in Lean against its own Python; and checked the Main Theorem, Theorem DW and Corollaries 1.4–1.7 against Smith forms it computed from the graphs ($n \leq 11$ for the 2-parts, $n \leq 9$ for the odd parts): 161 checks, 0 failures. Verdict: «holds as far as I can check»; no Lean statement says something other than the paper. It found four errors in the record, none in a statement: the folded-cube sentence of §1.3; the number of checks in Table 6a (C13, and C21–C30 left blank; the author then found that C1–C16 report counts too); 885, not 884, lines of `#print axioms`; and 3 449 theorem and lemma declarations, not 3 237 (the latter counts lines). They are corrected here. What it did not check: 408 of the 416 modules were not re-elaborated by it, 59 of the 68 rows of the §3 table were not read statement by statement, and the provenance data of Table 6a. Its report and checks are kept with the record (`reports/LECTOR_FRIO_LEAN_1/delivered/`).
8. **Aristotle's per-task times** are shown on its web page and were not exported. The download times in §6 are the save times of the result files. Aristotle's own account of its 68 runs is in `ARISTOTLE_SUMMARY.md` of the returned project, one section per run.

---

3. How the proof is organised in Lean

The paper was cut into **68 pieces**, `C1–C52` (with letter suffixes where one section needed several pieces). Each piece is one short, self-contained Markdown file `lean/C*.md`, written from version 8 of the paper. Each was sent to Aristotle (Harmonic) in the project `968c28ea-180f-4dbd-a11f-1f97b45d6d95`, which returned the whole Lean project with one new folder. The order was bottom-up: the floor and ceiling of the Smith form (§6–§7), then the dance (§4), then the group ring (§2–§3), then the Main Theorem (§8), the consequences (§9), the fold law (§10) and the odd parts.

No piece modified a file of an earlier piece, and the compiled project is the one Aristotle returned. Checked on 11 October 2026:

- the project's `RequestProject/` folder is identical, file by file, to the one in Aristotle's last returned project (piece C52);
- every one of the 68 folders is identical to its copy in the returned project of the piece that created it;
- `lakefile.toml`, `lean-toolchain` and `lake-manifest.json` are identical to Aristotle's;
- the 68 piece files inside the returned project are identical to the ones sent.

Piece	Paper v8	Lean folder	Main Lean results
C1	§7.1: Lemmas 7.1–7.3	`AntitonicFit`	`existsUnique_fit`, `lemma_7_1`, `lemma_7_2_a`, `lemma_7_2_b1`, `lemma_7_2_b2`, `lemma_7_2_b3`, `lemma_7_2_c`, `lemma_7_3_fit`, `lemma_7_3_ifit`
C2	Lemma 6.1 (tropical bound), the unique least term, Lemma 6.4, Theorem 7.4 (Theorem U)	`SmithBounds`	`lemma_A1`, `lemma_B`, `corollary_B'`, `lemma_6_4`

Piece	Paper v8	Lean folder	Main Lean results
C3	§5: Lemma 5.1, Propositions 5.2–5.4	`Rulers`	`lemma_5_0_xi`, `lemma_5_1_a`, `lemma_5_1_b`, `proposition_5_2`, `proposition_5_3`
C4	Theorem 6.2 (Theorem O), Corollary 6.3	`GoldenMatching`	`theorem_A`, `theorem_B`, `corollary_D`
C5	§6.1 at the prime 2, Theorem 6.5	`Divisors`	`cauchy_binet`, `dd_top`, `corollary_A3_convex`
C6	§7.3: Lemmas 7.5–7.7	`Houses`	`lemma_A`, `lemma_A'_cost`, `lemma_B1`, …
C7	Theorem 7.8 (Theorem F), steps 1–3	`TheoremF`	`lemma_0a`, `lemma_0b`, `lemma_0c_existsUnique`, `lemma_0d`, `lemma_0e`, `lemma_0f_J0`; the walk and the strict cut
C8	Theorem 7.8, steps 4–8 (the potential)	`Potential`	`lemma_P1_*`, `lemma_P2`–`lemma_P4`
C9	Lemma 7.9, Theorem 7.10 (house form)	`CellFloor`	`lemma_7_9_I1`, `lemma_7_9_I2`, `lemma_7_9_I3`, `theorem_B`
C10	§4.4–§4.5, Lemma 4.5	`Deficit`	`lemma_4_5`, `A3_deficit`
C11	§4.1, Proposition 4.1	`Payment`	`theorem_A`, `corollary_A1`, `corollary_A2`
C12	Proposition 4.2	`Split`	`theorem_B`, `corollary_B1`
C13	Proposition 4.3	`Dance`	`D1_scaled_dance`, `D3_final_matrix`
C14	Theorem 4.4, Theorem D	`Windows`	`theorem_4_4`, `theorem_D_i`, `theorem_D_ii`
C15	Theorem 7.10 for the dance matrix	`DanceSmith`	`P1`, `P2`, `Q1`, `Q2`, `S1`–`S3`
C16	§8 per family	`FamilyGroup`	`T1`, `T2`, `U1`, `U2`, `X1`, `X2`
C17	§2.1, Lemma 2.1	`CubeLaplacian`	`A1`–`A4`, `B1`, `B2`, `C1`
C18	§2.2–§2.3, Lemma 2.2, Corollary 2.3	`DPLattice`	`T1`–`T4`, `T5_delta_zero`, `T5_nabla_zero`
C19	Lemma 2.4	`WeylFilt`	`M1_*`, the four maps
C20	Proposition 3.1 (a), (b), (d)	`Doubling`	`A1`–`A3`, `B1`, `B2`
C21	Proposition 3.1 (c)	`Twist`	`R0`–`R3`, `T0`–`T2`
C22	Lemma 3.5, first sentence	`Lemma35`	`S1`, `S2`, `L1`
C23	tensor products (toward Lemma 3.5)	`Tensor2`	`tensor_isDP`, `P1`–`P3`
C24	Lemma 3.5, second sentence	`Lemma35b`	`I1`–`I3`, `B1`–`B3`, `F1`, `F2`
C25	Lemma 3.2 (Lemma W), Lemma 3.3 for one pair	`LemmaW`	`P1`–`P3`, `D1`–`D4`
C26	Lemma 3.3 for one pair, two forms	`LiftExt`	`L1`, `X1`, `B1`–`B4`
C27	Lemma 3.3, second sentence	`Cocycle`	`G2`–`G5`, `G6a`–`G6c`, `G7`
C28	lifting modulo 2, Corollary 3.4	`Lift2`	`D1`–`D3`, `L0`–`L2`, `C1`, `C2`

Piece	Paper v8	Lean folder	Main Lean results
C29	filtrations, the dance lattices $T(\lambda)$	`Filt`	`S1`, `S2`, `I1`, `I2`, `E1–E3`, `V1`
C30	the step of Theorem 3.6	`Step36`	`W1`, `W2`, `A1`, `K1–K3`, `K4a–K4e`
C31	Theorem 3.6, Lemma 3.7	`FamDec`	`M3`, `M4`, `P1`, `P2`, `D1–D3`
C32	Theorem 3.8	`Families`	`B0b`, `B1`, `B2`, `Q0`
C33	§8: the Main Theorem	`MainTheorem`	**`M1`**, `M2`, `G2`, `M3_*`
C34	Lemma 9.1, Corollary 1.5	`BaiCounts`	**`B1`**, **`B2`**, `A1`, `A2`, `A3_pos`, `F1`, `F2`
C35	Lemmas 9.2, 9.5, Proposition 9.6 (M), (K)	`Podium`	`P0`, `La`, *`Lb`*, `Lc_Y_eq_clear`
C35b	Propositions 9.3, 9.4, 9.6	`PodiumWalk`	`H1`, `H2`, `T1–T3`, `Z1`, `Z2`
C36	Lemma 9.7, Corollary 1.6	`PodiumCor`	**`P1–P3`**, **`G1–G3`**, `E1–E3`, `L1–L3`
C37	Lemmas 9.8, 9.9, Theorem 9.10, Corollary 1.7	`Mirror`	**`Y2`**, `A1–A5`, `B1`, `B2`
C38	Lemma 10.1, Corollary 10.2 (module form)	`Fold`	`G1–G3`, `push_lap`
C38b	Corollary 10.2 as groups, Proposition 10.3	`FoldGroup`	`KBar_finite`, `cokerBarEquivKBar`, `K1–K3`, `P2`
C39	Lemmas 10.4(a), 10.5(a)	`FoldHalf`	`S1–S4`, `E0–E3`
C39b	Lemmas 10.4(b), 10.5(b)	`FoldCube`	`O1–O5`, `V1`
C40	§10.3: the formal operators	`FormalOps`	`Bf.instRing`, `Bf.instAlgebra`, `B2–B4`, `T0`, `T1`
C40b	Lemma 10.6	`FormalDance`	`C1–C4`, `L1–L3`
C40c	one move of the dance (Proposition 10.7, one step)	`FormalAct`	`N1`, `N2`, `A1–A4`
C40d	Proposition 10.7	`FormalProp`	`Q0–Q4`
C41	Lemma 10.8	`InvBlock`	`V1–V5`
C42	Corollary 10.9	`InvFloors`	`F1–F6`, `C109`
C43	Lemma 10.10	`PathSum`	`P0–P2`, `PathSum.R`
C44	Lemma 10.11, Theorem 10.12 (first half, $\eta = 1$)	`Stability`	`S0–S2`, `A1–A3`, `F1`, `F2`
C45	Theorem 10.12 (first half, every unit η)	`Conjugation`	`C1–C5`, `D0`, `Conjugation.I`, `X1`
C46	Theorem 10.12 (second half)	`Smith2`	`M1–M5`, `Z0–Z2`
C47	Lemmas 10.13, 10.14	`RingK`	`K1–K6`, `S0`
C48	Theorem 10.15	`Cleanliness`	`W1–W5`, `V0–V2`
C48b	Theorem 10.16 for $c \geq 1$	`DanceXi`	`T1–T3`, `R0–R2`
C49	Corollary 10.17 for $j \geq 1$ and $l = 0$	`FoldOdd`	`F1–F6`, `C1`, `C2`
C49b	§10.6, first half	`ContZero`	`K1–K6`, `T1`, `T2`

Piece	Paper v8	Lean folder	Main Lean results
C49c	Theorem 10.16, Corollary 10.17 for every index	`ZeroSmith`	`G1`, `G2`, `Z3`, `V4–V7`, `R6`, `C5`
C50	Lemma 10.19	`GlueDance`	`B1–B4`, `G2`, `S1–S3`
C50b	Lemma 10.18	`DanceMod2`	`R1`, `R2`, `K1–K4`, `M1`, `M2`
C50c	the arms of the even families	`ArmsEven`	`P1–P3`, `A1`, `H1`, `H2`, `C1`, `E1`
C50d	Lemma 10.20	`Bend`	`N0–N2`, `Y0`, `Y1`, `Bend.W`, `B1`, `B2`
C50e	Corollary 10.21	`BendCor`	`A0–A2`, `F1`, `F2`, `W1–W3`
C50f	the glued system on *T(l)*	`GlueEven`	`S1`, `S2`, `O1`, `O2`, `Q1–Q4`
C50g	Theorem 10.22 for $i \geq 1$	`EvenFold`	`P0`, `P1`, `M1`, `M2`, `EvenFold.A`, `EvenFold.T`
C50h	Theorem 10.22 for every *i*	`EvenZero`	`M0`, `M1'`, `K1–K3`, `EvenZero.L`, `T0`, `TT`
C51	Theorem DW, §10.8	`FoldLaw`	**`FoldLaw.H`**, **`DW`**, **`DW'`**, `L0`, **`L1`**, **`FoldLaw.R`**
C52	the odd parts, Corollary 1.4	`OddParts`	**`OddParts.B1`**, **`C5`**, `Q0–Q2`, `OddParts.X`, `F1–F3`, `B0`

The third column lists the folder (and namespace) and the fourth the main results of each piece, among those whose axioms its check file prints. The names in bold are statements printed in §1.1. Every one of the 68 numbered statements of §2–§10 of the paper is in a piece of this table.

3.1 The chain of the Main Theorem

Every arrow is a Lean proof.

1. **The group ring** (C17). The Laplacian of Q_n is multiplication by $\sigma = \Sigma_i (1 - x_i)$ in $\mathbb{Z}[(\mathbb{Z}/2)^n]$, with $\sigma = U + 2D$ in the basis s_S (Lemma 2.1). $coker = \mathbb{Z} \oplus K(Q_n)$, and over $\mathbb{Z}_2$, $\mathbb{Z}_2 \oplus Syl_2\, K(Q_n)$ is the cokernel of σ.
2. **The dictionary and the family decomposition** (C18–C31):
   - the divided-power lattices and the dictionary $\sigma = F + n - H$ (Lemma 2.2, Corollary 2.3);
   - the doubling functor Φ (Proposition 3.1);
   - lifting from $\mathbb{F}_2$ (Lemmas 3.2, 3.3, Corollary 3.4), Lemma 3.5;
   - hence $V^{\otimes n} \cong \oplus_\lambda T(\lambda)^{m_\lambda(n)}$ over $\mathbb{Z}_2$ (Theorem 3.6), compatibly with σ.
3. **The groups of the families** (C32): $\mathbb{Z}_2 \oplus Syl_2\, K(Q_n) \cong \oplus_\lambda X^{(1)}(\lambda, (n-\lambda)/2)^{m_\lambda(n)}$, where $X^{(1)}$ is the cokernel of σ on the dance lattice (Theorem 3.8).
4. **The dance** (C10–C14): the twist and split moves, the dance, the windows, so $X^{(1)}(\lambda, i)$ is the small clocks plus $coker(2^k M(\lambda, i))$ (Theorem D).
5. **The Smith form of *M*** (C1–C9, C15): the ceiling through the unique golden matching (Theorem O) and the floor through the potential (Theorem F), so the Smith exponents of M are the pooled clocks (Theorem 7.10).
6. **The family group** (C16, C33 `G2`): $X^{(1)}(\lambda, i) \cong X(\lambda, i)$ with the rule of §1.2, as $\mathbb{Z}_2$-modules.
7. **The Main Theorem** (C33 `M1`).

3.2 The chain of the fold law and of the consequences

- **Corollaries 1.5–1.7** (C34–C37) are read off the Main Theorem: Bai's counts from the numbers of factors of each family; the podium from the largest clocks; the mirror from a bijection between the families of $2^e - 1$ and those of 2^e.

- **Theorem DW** (C38–C51):
  - the antipode on each family (Lemma 10.1) splits the folded group family by family (Corollary 10.2, Proposition 10.3);
  - the formal dance (Proposition 10.7) and rigidity (Theorem 10.12) show that the perturbation of the folded operator changes no valuation;
  - cleanliness (Theorem 10.15) holds through the ring $\mathcal{K}$;
  - the odd families (Corollary 10.17) and the even families (Theorem 10.22) follow;
  - Proposition 10.3 assembles them into `DW`.
- **The odd parts** (C52) by an integer Fourier transform on $(\mathbb{Z}/2)^n$. The Laplacian eigenvalues of Q_n are $2|S|$ for $S \subseteq \{1, \ldots, n\}$; those of the folded cube are $2|S|$ for $|S|$ even; inverting the transform costs only powers of 2.

3.3 The dependency cone

A short Lean program (`checks/DepsCubo.lean`, log `logs/DepsCubo_postclean.log`) collects every declaration that a theorem uses, directly or indirectly, and keeps those defined in the project. Its output is identical, line for line, before and after the clean rebuild (§8), except the watchdog's final line (time and memory).

Theorem	Project declarations in its cone	Project modules in its cone
`MainTheorem.M1` (the Main Theorem)	4 255	161
`Windows.theorem_D_i` (Theorem D)	262	17
`BaiCounts.B2` (Corollary 1.5)	4 412	166
`PodiumCor.G1` (Corollary 1.6)	5 036	181
`Mirror.Y2` (Corollary 1.7)	4 522	168
`FoldLaw.DW` (Theorem DW)	6 452	297
`FoldLaw.DW'` (its equivalent form)	6 771	310
`FoldLaw.R` (the mod-2 layer)	6 749	310
`OddParts.B1` (Bai's odd part)	196	10
`OddParts.C5` (the odd part of $\bar{K}(n)$)	277	13
union of the sixteen headline theorems	**7 988**	**341**

The counts include the auxiliary declarations Lean generates (equation lemmas, `match` functions, proof terms). The union is over the sixteen theorems `M1`, `M2`, `theorem_D_i`, `B1`, `B2`, `G1`, `G2`, `G3`, `Y2`, `FoldLaw.H`, `DW`, `DW'`, `L1`, `FoldLaw.R`, `OddParts.B1` and `C5`. The project has 416 modules, and 75 are outside the union:

- the 67 summary modules `.All` of the folders, which only import and document;
- `Main`, which imports only Mathlib and sets options;
- seven side modules:
  - `AntitonicFit.Examples` (the worked examples of §7.1);
  - `FoldOdd.PartC` (an earlier form of Corollary 10.17, superseded by C49c);
  - `Houses.Windows` (Lemma 7.6 in a second form);
  - `Lemma35.PartL3` (Lemma 3.5 over $\mathbb{F}_2$ as stated in the paper, with the remark over $\mathbb{Z}$);

- `LiftExt.Basic`, `LiftExt.PartB`, `LiftExt.PartX` (Lemma 3.3 for one pair, in the lifting and extension forms; the route through cocycles of C27 is the one used).

The three cases `P1–P3` of Corollary 1.6 are not in the union list; they live in `PodiumCor.Corollary`, which is in the union.

---

4. Trust base

- **What must be trusted:**
  - the Lean 4 kernel, toolchain `leanprover/lean4:v4.28.0`;
  - Mathlib at commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365` (tag `v4.28.0`, `lake-manifest.json`);
  - the three standard axioms `propext`, `Classical.choice`, `Quot.sound`;
  - the audit of the statements (§1.3).
- **What need not be trusted:** Aristotle (Harmonic), the AI system that wrote the Lean proofs, and the author's brute-force checks. Every module was compiled again on the author's machine, and the kernel re-checks every proof.
- **Forbidden constructs**, searched in all 416 files: none.
  - No `sorry`, no `admit`, no `opaque`, no `unsafe`, no `partial def`, no `@[extern]`, no `implemented_by`, no `native_decide`.
  - The word `axiom` occurs seven times, all inside comments («the axioms (W), (P0), (P1), (K)» of a divided-power lattice, a structure of the project). No axiom is declared.
- **The tactic `decide`** occurs in 16 files. It is used for small finite facts: parities, inequalities of natural numbers, and the values of the worked examples in `AntitonicFit/Examples.lean`. It is kernel-checked and adds no axiom, as `#print axioms` confirms.
- **Options.**
  - Resource limits: `maxHeartbeats` (in `Main.lean`, and raised locally in three files of `Lemma35`), `synthInstance.maxHeartbeats` (in `Main.lean` and in one file of `RingK`), `maxRecDepth` and `synthInstance.maxSize` (in `Main.lean`).
  - Printing options `pp.*` and elaboration options `autoImplicit false`, `relaxedAutoImplicit false` (in `Main.lean`), and `grind.warning false`.
  - `set_option hygiene false in` once, on a local tactic macro (`DPLattice/Weyl.lean`): it changes how the names inside the macro are resolved, not what the kernel accepts.
  - None affects soundness.
- **Instances.** The project declares 38 instances. They are of three kinds:
  - **ring and algebra structures on types of the project**, with every axiom of the structure proved: `Bf A` (the formal operators of §10.3); `RS`, the ring of §10.5 in which $\mathcal{K}$ lives, with coefficients in `RingK.S` (the ring $\mathcal{S}$, a trivial square-zero extension of $\mathbb{F}_2$ by $\mathbb{F}_2[[t]]$); `RA R`, the same ring with coefficients in R (Lemma 10.18); and the product structures of `XFam`;
  - **facts**: decidability, finiteness, `IsEmpty`, `IsCentralScalar`;
  - **two instances of the project's class `OddUnits`**, for `ℤ_[2]` and `ℚ_[2]`: a structure whose field is a proof that odd integers are units there.

Besides, one line `attribute [instance] DPLattice.fintype DPLattice.decEq` (`DPLattice/Basic.lean`) makes two more declarations instances: the finiteness and the decidable equality of the index type of a divided-power lattice. The project also has three tactic macros (`weyl_close`, local, in `DPLattice/Weyl.lean`; `wf_simp` and `wf_close` in `WeylFilt/Comm.lean`) and two local notations (`lV`, `lP` in `Split/ChangeOfBasis.lean`, abbreviations of `DirectSum.lof`). A macro only produces a proof term that the kernel still checks.

An instance adds no axiom. The structures on the types in the statements of §1.1 are the product structures of Mathlib (`XFam`, through `inferInstanceAs`).

- **Axioms**, printed after each piece (68 final check logs; for C31 the final run is `checkC31b.log`) and again in `CheckCubo_postclean.log` and `CertCubo.log`.
  - Over the 68 final check logs, 885 lines of `#print axioms`: 868 give the three axioms, 12 give `[propext, Quot.sound]`, 4 give `[propext]`, and one (`FormalDance.C1`, `checkC40b.log`) depends on no axiom. The last seventeen are small lemmas about digits, lists, ranks and units.
  - No line names any other axiom, and none contains `sorryAx`.
  - In `CheckCubo_postclean.log` the twenty-one statements of §1.4 give the three axioms.
- **Aristotle's own register** for the project: 527 properties PROVED, 0 NEGATED, 0 in progress, after the last piece (audit log `lean/INFORME_ARISTOTLE_CUBO_v1.md`, run C52).

---

5. The procedure

The cycle, for each of the 68 pieces, in one turn:

1. **Write and brute-force before sending.** A piece file is written from the paper. A brute-force script checks every statement of the piece on small cases before it is sent, with negative controls: deliberately wrong variants that must fail.
2. **Send** to Aristotle, in the same project.
3. **When it comes back:**
   - download the project;
   - compare it with the compiled one (only the new folder may differ);
   - search for forbidden constructs;
   - build the new modules locally, one at a time;
   - run a check file `checks/CheckC<N>.lean` that restates the main statements **by hand** and prints their axioms;
   - audit faithfulness;
   - register; then write the next piece.

One module at a time under a watchdog (`vigia.sh`). Every local `lake build` of a module ran alone, with a cap of 4.5 GB of real memory and 900 s, and the cap was never raised.

Stop conditions, fixed in advance: a property NEGATED by Aristotle; a statement that is not faithful; a build failure of Aristotle's code; a `sorry` or an extra axiom; a change to an old file. **None occurred.**

Errors along the way, all of the author, all recorded (audit log; logs kept with their names):

- **Check files.** Seventeen runs of check files failed through errors in the hand restatements, and were re-run after correction; Aristotle's code was never changed. Examples: a missing `open scoped Pointwise`, a cast, missing type ascriptions.

    - Fourteen of these runs have their logs kept: twelve first runs of piece check files (two for C10, one each for C4, C6, C11, C13, C14, C15, C17, C18, C19, C22); the first run of `CheckC31.lean`, whose rerun is `checkC31b.log`; and the first run of `CheckCubo.lean` (`logs/CheckCubo_run1.log`).
    - In three more (C28, C29, C50f) the rerun overwrote the log; the audit log records them.
- **Builds.** Three local builds were cut by a time limit of the author's own shell tool, not by the watchdog, and were resumed from the module where they stopped (C1, C27, C39).
- **Brute force.** Some brute-force scripts were corrected before sending (C1, C2); in three of them one control stayed silent, and this is declared in Table 6a (C28, C33, C49b).
- **One piece file was wrong** (C46, clause (M3)): it stated a clause without a hypothesis it needs. Aristotle added the hypothesis `OddUnits`, which the piece's (M2) already carried. The fix is right and the error was the author's.

«Review suggested». Aristotle's page showed this badge on three pieces (C25, C44, C46). In C25 and C44 it accompanied generality notes only: hypotheses dropped, giving statements more general than asked. In C46 it accompanied the correction just described. More generally, Aristotle declared several statements more general than the piece asked (for instance, in C1, (b2) without the hypotheses on the cuts); each is noted in the docstrings and in its run summary.

---

6. Data per piece

- **Task:** Aristotle's task (run) identifier, first 8 hex digits; project `968c28ea-180f-4dbd-a11f-1f97b45d6d95`.
- **Tarball:** the file returned by Aristotle, saved in `~/Downloads` as `968c28ea-…-aristotle (n).tar.gz`; *n* absent for C1; *(33)* is a second download of *(32)*, identical by md5; md5 first 8 hex digits; **downloaded** is its save time (CEST).
- **Brute force:** the script `lean/chkC<N>.py`, log `logs/chkC<N>.log`, every one with 0 failures. The number of checks is the total the log reports; where the log reports counts by kind only (C1–C16), it is their sum, with the number of kinds.
- **Files, lines, theorems, definitions:** of the folder, in the final project. The column «`theorem`/`lemma` lines» counts lines beginning with the word `theorem` or `lemma` (it leaves out the declarations preceded by an attribute such as `@[simp]`; the declarations are counted in §7); definitions are lines beginning with `def`, `abbrev`, `noncomputable def` or `noncomputable abbrev`.
- **Rebuild:** the clean rebuild of §8, summed over the folder's modules, and its largest real-memory peak (GiB = kB/1 048 576). These numbers come from one uniform run of the whole project, not from the builds made piece by piece.

The two tables are generated by `lean/CERTIFICADO/make_tables.py`: Table 6b from data files, except the four counts of the row of `Main.lean` (1 file, 24 lines, 0, 0), which are written into the script; and the checks column of Table 6a from the brute-force logs themselves. Two columns of Table 6a were transcribed by hand into that script: the Aristotle task identifiers, from the audit log, and the controls (fired/total), from the logs; the cold reader checked the controls against the 68 logs, with no mismatch.

Table 6a. Provenance.

Piece	Aristotle task	Tarball	md5	Downloaded	Brute force before sending
C1	2b8cfa39	—	ff90158b	06/10 17:21	1 591 122 checks in 10 kinds, 0 failures; controls 2/2
C2	e0b97d3b	(1)	581945f1	06/10 18:25	10 766 285 checks in 6 kinds, 0 failures; controls 5/5
C3	b15574a9	(2)	5236be3e	06/10 19:06	317 339 checks in 7 kinds, 0 failures; controls 3/3
C4	339a7b11	(3)	f234e506	06/10 20:04	10 000 checks in 4 kinds, 0 failures; controls 4/4
C5	83be8821	(4)	63bc35cb	06/10 20:47	6 829 checks in 4 kinds, 0 failures; controls 3/3
C6	194e2f27	(5)	47f5f293	06/10 21:35	12 745 256 checks in 19 kinds, 0 failures; controls 4/4
C7	a15e0203	(6)	89c426c3	06/10 22:09	177 666 checks in 16 kinds, 0 failures; controls 3/3
C8	13a49f40	(7)	bdb2c903	06/10 22:49	2 396 430 checks in 14 kinds, 0 failures; controls 5/5
C9	813c96e8	(8)	49b12b21	07/10 07:08	660 735 checks in 14 kinds, 0 failures; controls 4/4
C10	0855be32	(9)	6dcbfbfa	07/10 08:07	70 919 checks in 11 kinds, 0 failures; controls 5/5
C11	ad645d58	(10)	8cb3d104	07/10 09:02	1 500 checks in 1 kind, 0 failures; controls 4/4
C12	3f8eef5f	(11)	f7f3172f	07/10 10:44	1 500 checks in 2 kinds, 0 failures; controls 5/5
C13	8906e033	(12)	fe7ef3b8	07/10 11:40	552 checks in 4 kinds, 0 failures; controls 4/4
C14	a49f4cd1	(13)	d3b06072	07/10 12:48	24 060 checks in 6 kinds, 0 failures; controls 5/5
C15	6c933352	(14)	36bad99b	07/10 14:12	3 920 checks in 13 kinds, 0 failures; controls 3/3
C16	8cfae8e6	(15)	ea007ade	07/10 14:50	1 892 checks in 14 kinds, 0 failures; controls 4/4
C17	544ce1db	(16)	00dea20a	07/10 15:48	7 449 checks, 0 failures; controls 5/5
C18	a46e4881	(17)	d37ba88d	07/10 17:01	227 checks, 0 failures; controls 5/5
C19	3209f52c	(18)	5884c436	07/10 18:16	1 030 checks, 0 failures; controls 5/5
C20	a60cc4f4	(19)	05bdb8fb	07/10 19:36	2 238 checks, 0 failures; controls 4/4
C21	8d8b879c	(20)	faa29efe	07/10 20:40	22 196 checks, 0 failures; controls 6/6
C22	3f5c9bd7	(21)	42fbe074	07/10 21:38	7 checks, 0 failures; controls 5/5
C23	4cd90d1b	(22)	d617e869	07/10 22:38	347 checks, 0 failures; controls 5/5
C24	7a4cf945	(23)	7017cd4e	08/10 07:12	98 checks, 0 failures; controls 5/5
C25	aec9cbab	(24)	95de6f33	08/10 07:54	686 checks, 0 failures; controls 5/5
C26	4b5ee80c	(25)	d0018f75	08/10 08:48	2 088 checks, 0 failures; controls 5/5
C27	51e14db3	(26)	1eda4a7c	08/10 09:35	297 checks, 0 failures; controls 4/4
C28	848f2784	(27)	31ab42c4	08/10 10:25	582 checks, 0 failures; controls 2/3 (one silent, declared)
C29	c29eb434	(28)	5c3703fe	08/10 11:36	470 checks, 0 failures; controls 3/3
C30	dbbc5c96	(29)	17791127	08/10 12:37	56 checks, 0 failures; controls 3/3
C31	148c191c	(30)	ef0d093d	08/10 13:26	833 checks, 0 failures; controls 3/3
C32	7a8f97ec	(31)	a7a9bd89	08/10 15:04	2 343 checks, 0 failures; controls 3/3
C33	960dc443	(32)	48bb394c	08/10 15:54	2 407 checks, 0 failures; controls 3/4 (one silent, declared)

Piece	Aristotle task	Tarball	md5	Downloaded	Brute force before sending
C34	2456f22c	(34)	750c2824	08/10 16:45	53 945 checks, 0 failures; controls 4/4
C35	4f36edd7	(35)	8b95a075	08/10 17:28	337 528 checks, 0 failures; controls 4/4
C35b	ffecadc3	(36)	51e9d693	08/10 18:23	22 447 checks, 0 failures; controls 4/4
C36	bf28bdd8	(37)	b8ac7267	08/10 19:49	267 891 checks, 0 failures; controls 5/5
C37	dbf16242	(38)	9160dd05	08/10 21:09	83 102 checks, 0 failures; controls 6/6
C38	5fde1d99	(39)	a563c186	08/10 22:04	977 checks, 0 failures; controls 3/3
C38b	02a121a5	(40)	fdb3cb08	08/10 22:57	254 checks, 0 failures; controls 4/4
C39	f1cf204b	(41)	c27b9e5e	09/10 06:50	711 checks, 0 failures; controls 4/4
C39b	579e5944	(42)	f8a5c408	09/10 08:27	673 checks, 0 failures; controls 4/4
C40	0246c6df	(43)	76dc0932	09/10 09:29	773 checks, 0 failures; controls 6/6
C40b	afeddeb6	(44)	3930feae	09/10 10:45	412 checks, 0 failures; controls 6/6
C40c	84a3b2d3	(45)	4160be08	09/10 12:04	184 checks, 0 failures; controls 6/6
C40d	23cdce92	(46)	e944373f	09/10 13:09	493 checks, 0 failures; controls 5/5
C41	86ac37fe	(47)	de408b6d	09/10 14:27	334 checks, 0 failures; controls 5/5
C42	d4e089ce	(48)	22954370	09/10 15:38	528 checks, 0 failures; controls 5/5
C43	198fbc5c	(49)	bfcba602	09/10 18:46	19 713 checks, 0 failures; controls 4/4
C44	2cccc89d	(50)	834f25c5	09/10 19:32	11 994 checks, 0 failures; controls 5/5
C45	76de0979	(51)	c6e2efbd	09/10 20:35	1 391 checks, 0 failures; controls 4/4
C46	d36e2d4f	(52)	59e6e881	09/10 21:45	1 008 checks, 0 failures; controls 4/4
C47	19843fc1	(53)	12445e04	09/10 22:27	12 622 checks, 0 failures; controls 4/4
C48	556eee6a	(54)	de70d700	09/10 23:20	24 493 checks, 0 failures; controls 5/5
C48b	a5c4d914	(55)	7a2cee33	10/10 06:36	1 286 checks, 0 failures; controls 3/3
C49	1810c6c2	(56)	896a5cf5	10/10 07:37	288 checks, 0 failures; controls 4/4
C49b	c6cd8d5a	(57)	3ee8e48d	10/10 08:30	1 464 checks, 0 failures; controls 3/4 (one silent, declared)
C49c	8ef876e1	(58)	8cd76e52	10/10 09:42	2 437 checks, 0 failures; controls 5/5
C50	ec5fa07e	(59)	26627efa	10/10 10:55	4 114 checks, 0 failures; controls 4/4
C50b	753368cf	(60)	e929a54b	10/10 11:47	11 616 checks, 0 failures; controls 3/3
C50c	2759b7d5	(61)	b9630e80	10/10 12:51	10 968 checks, 0 failures; controls 4/4
C50d	3af52ebd	(62)	86aef806	10/10 13:46	35 887 checks, 0 failures; controls 4/4
C50e	27ec0578	(63)	5cb23221	10/10 15:42	8 253 checks, 0 failures; controls 6/6
C50f	d6aa1d91	(64)	eb4c82f6	10/10 17:26	750 checks, 0 failures; controls 4/4
C50g	e5577a86	(65)	4b1ed92c	10/10 18:27	1 490 checks, 0 failures; controls 4/4
C50h	3b91d694	(66)	77b375b1	10/10 19:34	302 checks, 0 failures; controls 3/3
C51	222eb7ce	(67)	b3b3e1be	10/10 20:48	66 checks, 0 failures; controls 3/3

Piece	Aristotle task	Tarball	md5	Downloaded	Brute force before sending
C52	`505f1414`	(68)	`4ded9e0d`	10/10 21:46	204 checks, 0 failures; controls 3/3

Table 6b. The Lean folders and their clean rebuild.

Piece	Folder	Files	Lines	`theorem`/`lemma` lines	Definitions	Modules	Rebuilds	Peak GiB
C1	`AntitonicFit`	8	1 645	99	23	8	565	2.18
C2	`SmithBounds`	6	648	28	10	6	372	1.99
C3	`Rulers`	6	852	45	21	6	385	1.83
C4	`GoldenMatching`	5	897	38	11	5	317	1.96
C5	`Divisors`	6	770	45	5	6	382	1.83
C6	`Houses`	5	1 015	55	14	5	296	1.90
C7	`TheoremF`	5	968	43	7	5	313	1.91
C8	`Potential`	6	993	35	9	6	364	1.89
C9	`CellFloor`	4	524	21	10	4	246	1.85
C10	`Deficit`	6	620	22	5	6	358	2.10
C11	`Payment`	5	600	12	19	5	314	1.85
C12	`Split`	7	1 225	38	30	7	393	1.99
C13	`Dance`	7	873	24	29	7	405	2.06
C14	`Windows`	8	1 128	46	7	8	504	2.04
C15	`DanceSmith`	7	1 680	79	14	7	412	2.02
C16	`FamilyGroup`	6	532	23	6	6	356	1.96
C17	`CubeLaplacian`	8	951	57	31	8	487	2.13
C18	`DPLattice`	7	1 705	114	43	7	454	1.99
C19	`WeylFilt`	7	1 089	70	26	7	443	1.91
C20	`Doubling`	12	2 148	165	40	12	707	2.19
C21	`Twist`	6	1 033	72	17	6	402	2.01
C22	`Lemma35`	8	893	58	39	8	552	2.46
C23	`Tensor2`	7	1 147	74	17	7	502	1.85
C24	`Lemma35b`	2	868	54	24	2	89	2.06
C25	`LemmaW`	5	1 002	58	16	5	278	2.20
C26	`LiftExt`	4	575	29	1	4	242	1.91
C27	`Cocycle`	7	1 655	84	26	7	526	1.99
C28	`Lift2`	5	464	20	4	5	379	1.97
C29	`Filt`	6	1 077	63	9	6	356	1.97
C30	`Step36`	4	548	26	17	4	236	1.94

Piece	Folder	Files	Lines	theorem/lemma lines	Definitions	Modules	Rebuilds	Peak GiB
C31	FamDec	4	517	36	4	4	217	1.93
C32	Families	6	1 108	50	21	6	356	2.14
C33	MainTheorem	4	420	28	7	4	228	1.99
C34	BaiCounts	6	987	68	6	6	301	2.02
C35	Podium	5	779	53	5	5	268	1.95
C35b	PodiumWalk	6	1 073	49	1	6	345	2.03
C36	PodiumCor	11	1 910	127	8	11	638	1.98
C37	Mirror	7	1 463	67	11	7	376	2.06
C38	Fold	6	922	59	22	6	351	1.94
C38b	FoldGroup	5	554	22	20	5	306	1.97
C39	FoldHalf	7	1 381	110	30	7	423	1.93
C39b	FoldCube	8	1 182	75	25	8	523	1.86
C40	FormalOps	7	992	49	18	7	443	2.07
C40b	FormalDance	6	1 074	96	17	6	337	1.96
C40c	FormalAct	6	636	22	11	6	382	1.93
C40d	FormalProp	5	521	11	7	5	344	1.80
C41	InvBlock	4	529	17	8	4	256	1.84
C42	InvFloors	6	857	43	6	6	346	1.97
C43	PathSum	5	516	19	5	5	292	2.03
C44	Stability	7	1 032	53	9	7	345	1.95
C45	Conjugation	8	1 005	53	7	8	471	1.93
C46	Smith2	7	1 221	60	3	7	410	1.99
C47	RingK	6	1 144	69	14	6	399	2.06
C48	Cleanliness	6	820	43	14	6	345	1.96
C48b	DanceXi	3	239	8	3	3	172	1.86
C49	FoldOdd	5	455	22	11	5	277	2.01
C49b	ContZero	6	760	61	2	6	344	2.00
C49c	ZeroSmith	7	973	20	1	7	429	1.95
C50	GlueDance	6	668	44	6	6	354	1.97
C50b	DanceMod2	6	783	41	6	6	366	1.98
C50c	ArmsEven	7	471	20	3	7	348	2.09
C50d	Bend	7	715	22	1	7	399	2.02
C50e	BendCor	6	524	17	3	6	342	2.03

Piece	Folder	Files	Lines	theorem/lemma lines	Definitions	Modules	Rebuilds	Peak GiB
C50f	GlueEven	5	382	13	3	5	311	1.91
C50g	EvenFold	5	314	8	0	5	313	1.97
C50h	EvenZero	6	381	11	1	6	358	1.98
C51	FoldLaw	6	534	22	3	6	372	1.90
C52	OddParts	7	973	52	24	7	385	1.98
—	Main.lean	1	24	0	0	1	129	1.83
Total	68 folders + Main.lean	**416**	**60 964**	**3 237**	**876**	**416**	**25 236**	**2.46**

---

7. Totals

	Files	Lines	theorem/lemma lines	Definitions
The 68 folders	415	60 940	3 237	876
Main.lean (imports Mathlib, sets options)	1	24	0	0

- **Theorem and lemma declarations in code: 3 449.** The column above counts lines that begin with the word theorem or lemma; it leaves out the 213 declarations preceded by an attribute (@[simp] 210, @[ext] 3) and counts one comment line.
- **sorry: 0.**
- **Brute-force checks before sending:** 68 scripts, 0 failures. Every script carried negative controls; all fired except one control in each of C28, C33 and C49b, declared.
- **Calendar:** piece C1 sent on 6 October 2026 at about 16:30 CEST; the result of piece C52 downloaded on 10 October at 21:46 CEST, about four days and five hours for the 68 pieces.
- **Aristotle's register:** 527 properties PROVED, 0 NEGATED, 0 in progress.

---

8. Reproducibility

Machine: Apple Mac (arm64), 8 CPU cores, 8 GB of memory, macOS 26.5.2 (Darwin 25.5.0).

Toolchain: Lean leanprover/lean4:v4.28.0; Mathlib v4.28.0, at the commit

```
8f9d9cff6bd728b17a24e163c9402775d9e6a365
```

Paths. Every path in this certificate is relative to the project folder of the author's working tree; the public repository keeps the same relative paths (lean/…, logs/…, checks/…).

Commands, from the project root (lean/proyecto_cubo/ here):

```
lake exe cache get                      # Mathlib's compiled files
lake build                              # the 416 modules of RequestProject (or one at a time)
lake env lean checks/CheckCubo.lean     # the hand restatements of §1.4
```

```
lake env lean checks/CertCubo.lean       # the statements and definitions printed in §1
lake env lean checks/DepsCubo.lean       # the dependency cone of §3.3
```

On a machine with little memory, build the modules one at a time, in the order of `lean/clean_rebuild/orden_modulos.txt`, as the author did.

Clean rebuild. On 10 October 2026 the project's own compiled files (`.lake/build`) were moved aside, unchanged, to `.lake/build_pre_clean_2026-10-10`; Mathlib's compiled files (`.lake/packages`) were kept. Then every module of the project was compiled again from its source:

- one module at a time, each with its own `lake build` under the watchdog with the cap of §5;
- in an order where every module comes after all the modules it imports (`lean/clean_rebuild/orden_modulos.txt`).

The script and its logs are in `lean/clean_rebuild/`: `clean_rebuild_cubo.sh`, `clean_rebuild_2026-10-10.log`, and one log per module, `cr_logs/<module>.log`. The result:

- **All 416 modules were built, all with exit code 0, none skipped**, from 22:16:14 on 10 October to 05:20:33 on 11 October 2026 CEST (7 h 4 min). The module builds sum to 25 236 s; the longest is `Lemma35.L1Comm`, 192 s.
- The largest real-memory peak was **2.46 GiB** (`Lemma35.All`), about half the cap of 4.5 GiB.
- Every one of the 416 compiled `.olean` files is newer than the start of the rebuild.
- The sources did not change during the rebuild: the md5 of each of the 489 files of the manifest taken before it (`lean/clean_rebuild/MANIFEST_sources_md5.txt`: the 416 sources, the check files and the three lake files) is the same after it.

Then `CheckCubo.lean` and `DepsCubo.lean` were run again against the rebuilt files: logs `CheckCubo_postclean.log` (213 s, peak 1.56 GiB) and `DepsCubo_postclean.log` (298 s, peak 1.56 GiB), both with exit code 0. **Their output is identical, line for line, to the runs before the rebuild** (`CheckCubo.log`, `DepsCubo.log`), except the watchdog's final line (time and memory). `CertCubo.lean` was written and run after the rebuild (165 s, peak 1.50 GiB, exit code 0); it proves nothing and only prints.

Fingerprints of the source:

- The md5 of every one of the 416 `.lean` files of `RequestProject/` is listed in `lean/clean_rebuild/MANIFEST_md5_416.txt` (paths sorted bytewise). The md5 of that list is `cb0e4f7ad0bd315e3394c28c02a6321f`.
- `lakefile.toml d3ae68ec…`, `lean-toolchain b8b2923c…`, `lake-manifest.json 235e6523…`; the same three files as in the Lean project of the Chaise Longue.

---

9. What this certificate claims, in one paragraph

A computer proof assistant, Lean 4 with the Mathlib library, has checked a complete proof of the following statements, with no gap, no cited result and no axiom beyond the three standard ones:

> For every $n \geq 1$, the 2-part of the sandpile group of the n-dimensional hypercube, with one copy of the 2-adic integers added, is isomorphic to the direct sum of the explicit groups $X(\lambda,\ (n-\lambda)/2)$, each with the explicit multiplicity $m_\lambda(n)$. The 2-part of the sandpile group of the folded cube is the subgroup of doubles of that of the cube. And the consequences: Bai's counts and odd part, the $n+$

> *1* largest cyclic factors, Gao et al.'s Conjectures 4.14 and 5.4, the factor of the 2019 poster, and the odd part of the folded cube.

The proofs in Lean were written by the AI system Aristotle (Harmonic) from 68 pieces written from the paper. The audit — by the author, assisted by Claude (Anthropic) — checked that the statements match the paper, and compiled and re-checked everything on the author's machine. That audit, §1.2–§1.3 above, is the one part a reader must check.

References

- [Bai03] H. Bai, On the critical group of the n-cube, *Linear Algebra Appl.* **369** (2003), 251–261.
- [GMMY24] J. Gao, J. Marx-Kuo, V. McDonald, C. H. Yuen, Sandpile groups of Cayley graphs of $\mathbb{F}_2^r$, *Comm. Algebra* **52** (2024), 4459–4479; arXiv:1912.06919v3.
- [MGM19] J. Marx-Kuo, J. Gao, V. McDonald, *The sandpile group of Cayley graphs*, poster, Joint Mathematics Meetings, Baltimore, January 2019.
- [ABERS55] M. Ayer, H. D. Brunk, G. M. Ewing, W. T. Reid, E. Silverman, An empirical distribution function for sampling with incomplete information, *Ann. Math. Statist.* **26** (1955), 641–647.
- [BBBB72] R. E. Barlow, D. J. Bartholomew, J. M. Bremner, H. D. Brunk, *Statistical Inference under Order Restrictions*, Wiley, 1972.
- R. Amichis Luengo, *The Dancing Sand Theorem — The 2-part of the sandpile group of the hypercube, for every n*, version 8, Zenodo, DOI 10.5281/zenodo.23188299; all versions, concept DOI 10.5281/zenodo.23188298.
- R. Amichis Luengo, *The Dancing Sand Theorem — working record*, https://github.com/tretoef-estrella/dancing-sand-theorem.
- The Lean 4 theorem prover, https://lean-lang.org; the Mathlib library, https://github.com/leanprover-community/mathlib4.
- Aristotle, Harmonic, https://aristotle.harmonic.fun.